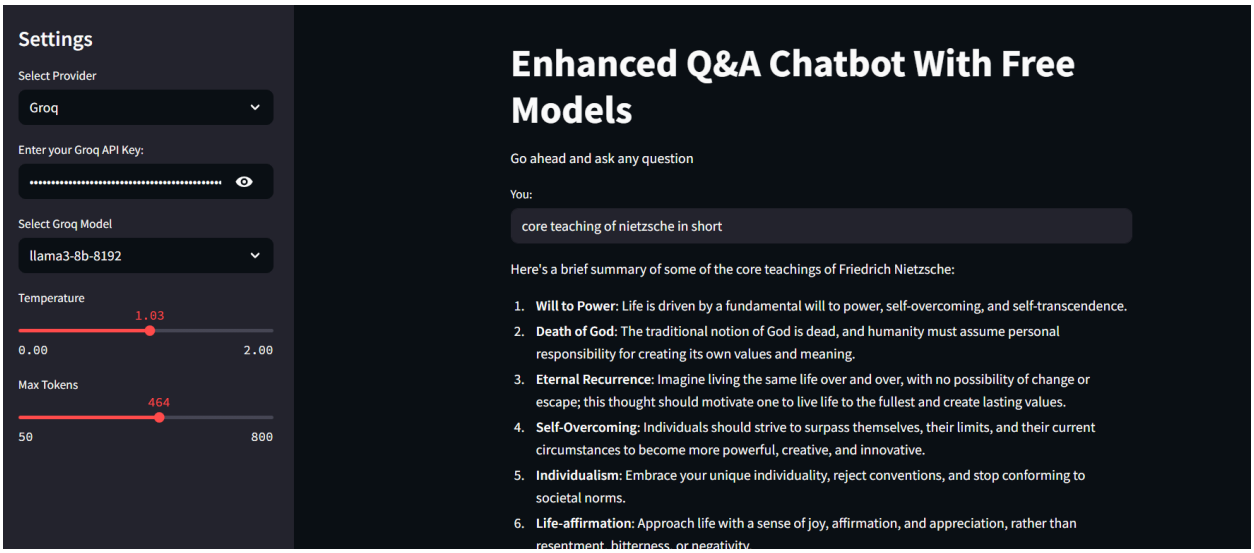
Simple ChatBot

Our app would look like:



1. Import Libraries

```
import streamlit as st
from langchain_groq import ChatGroq
from langchain_huggingface import HuggingFaceEndpoint
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
import os
from dotenv import load_dotenv
```

2. Langsmith Tracking (Optional)

```
# LangSmith Tracking (optional, keep if you use LangSmith)
os.environ["LANGCHAIN_API_KEY"] = os.getenv("LANGCHAIN_API_KEY")
```

```
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "Simple Q&A Chatbot"
```

🧠 3. Defining the Prompt Template

```
# Prompt Template
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant. Please respond to the user queries"),
        ("user", "Question: {question}")
```

🔄 4. Generating Responses

```
def generate_response(question, api_key, provider, model, temperature, max_tokens):
    """
    Generate response using either Groq or Hugging Face based on provider selection.
    """
    if provider == "Groq":
        llm = ChatGroq(
            model_name=model,
            api_key=api_key,
            temperature=temperature,
            max_tokens=max_tokens
        )
    else:  # Hugging Face
        llm = HuggingFaceEndpoint(
            endpoint_url=f"https://api-inference.huggingface.co/models/{model}",
            huggingfacehub_api_token=api_key,
            temperature=temperature,
            max_new_tokens=max_tokens
```

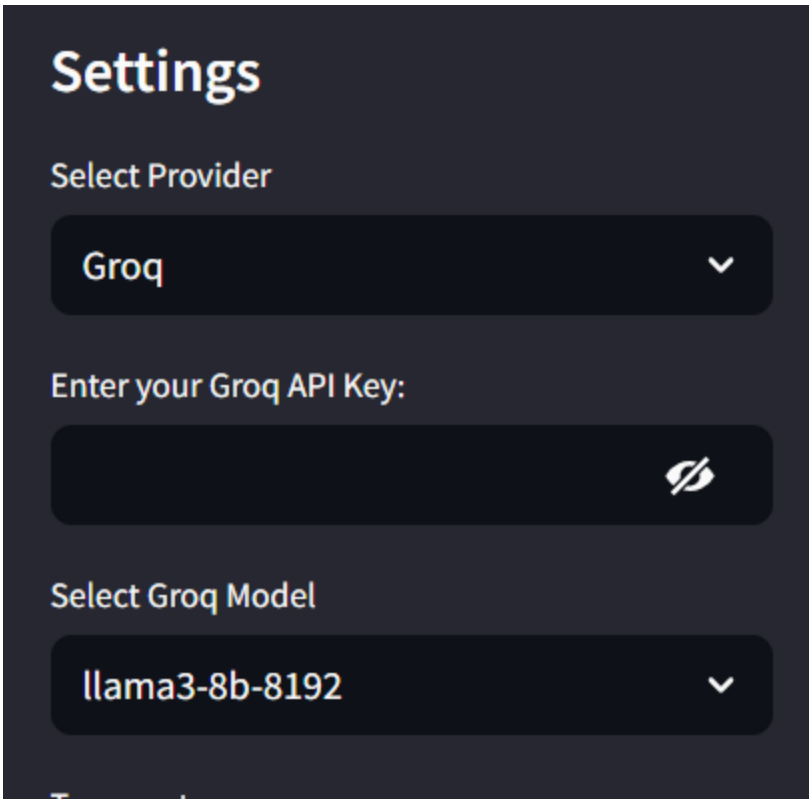
```
    )

    output_parser = StrOutputParser()
    chain = prompt | llm | output_parser
    answer = chain.invoke({'question': question})
    return answer
```

💡 **The function takes 6 arguments.**

🖼️ 5. Streamlit Application Interface

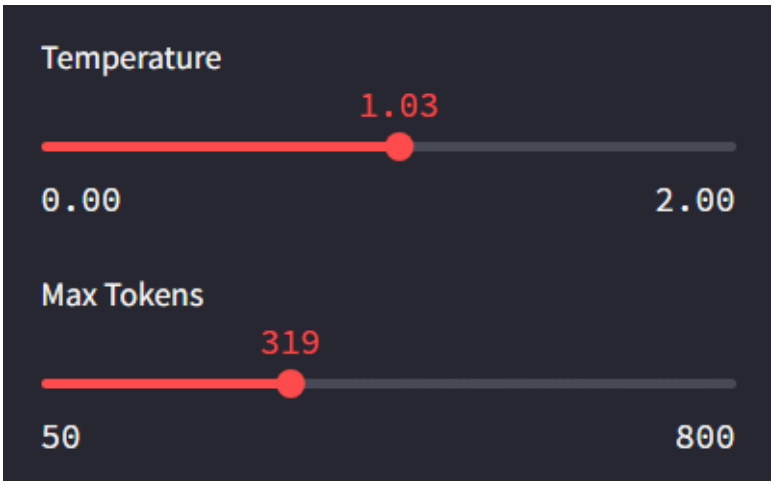
```
# Title of the app
st.title("Enhanced Q&A Chatbot With Free Models")

# Sidebar for settings
st.sidebar.title("Settings")

# Select provider (Groq or Hugging Face)
provider = st.sidebar.selectbox("Select Provider", ["Groq", "Hugging Face"])

# Input API key based on provider
if provider == "Groq":
    api_key = st.sidebar.text_input("Enter your Groq API Key:", type="passwor
d")
else:
    api_key = st.sidebar.text_input("Enter your Hugging Face API Key:", type="p
assword")

# Select model based on provider
if provider == "Groq":
    model = st.sidebar.selectbox("Select Groq Model", ["llama3-8b-8192", "mis
tral-saba-24b", "gemma2-9b-it"])
```

```
else:
    model = st.sidebar.selectbox("Select Hugging Face Model", [
        "mistralai/Mixtral-8×7B-Instruct-v0.1",
        "meta-llama/Llama-3.2-11B-Vision-Instruct",
        "google/flan-t5-large"
    ])
```



Add temperature & Max Tokens:

```
# Adjust response parameters
temperature = st.sidebar.slider("Temperature", min_value=0.0, max_value=2.0, va
max_tokens = st.sidebar.slider("Max Tokens", min_value=50, max_value=800, va
```



💬 6. Main Interface for User Interaction

```
# Main interface for user input
st.write("Go ahead and ask any question")
user_input = st.text_input("You:")
```

✅ 7. Handling User Input and Generating Response

```
# Handle user input and API key validation
if user_input and api_key:
    try:
        response = generate_response(user_input, api_key, provider, model, tem
perature, max_tokens)
        st.write(response)
```

```
    except Exception as e:
        st.error(f"Error: {str(e)}")
elif user_input:
    st.warning(f"Please enter the {provider} API Key in the sidebar")
else:
    st.write("Please provide the user input")
```

Entire Code:

```
import streamlit as st
from langchain_groq import ChatGroq
from langchain_huggingface import HuggingFaceEndpoint
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# LangSmith Tracking (optional, keep if you use LangSmith)
os.environ["LANGCHAIN_API_KEY"] = os.getenv("LANGCHAIN_API_KEY")
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "Simple Q&A Chatbot"

# Prompt Template
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant. Please respond to the user querie
s"),
        ("user", "Question: {question}")
    ]
)
```

```
def generate_response(question, api_key, provider, model, temperature, max_
tokens):
    """
    Generate response using either Groq or Hugging Face based on provider se
lection.
    """
    if provider == "Groq":
        llm = ChatGroq(
            model_name=model,
            api_key=api_key,
            temperature=temperature,
            max_tokens=max_tokens
        )
    else:  # Hugging Face
        llm = HuggingFaceEndpoint(
            endpoint_url=f"https://api-inference.huggingface.co/models/{model}",
            huggingfacehub_api_token=api_key,
            temperature=temperature,
            max_new_tokens=max_tokens
        )

    output_parser = StrOutputParser()
    chain = prompt | llm | output_parser
    answer = chain.invoke({'question': question})
    return answer

# Title of the app
st.title("Enhanced Q&A Chatbot With Free Models")

# Sidebar for settings
st.sidebar.title("Settings")

# Select provider (Groq or Hugging Face)
provider = st.sidebar.selectbox("Select Provider", ["Groq", "Hugging Face"])
```

```
# Input API key based on provider
if provider == "Groq":
    api_key = st.sidebar.text_input("Enter your Groq API Key:", type="passwor
d")
else:
    api_key = st.sidebar.text_input("Enter your Hugging Face API Key:", type="p
assword")

# Select model based on provider
if provider == "Groq":
    model = st.sidebar.selectbox("Select Groq Model", ["llama3-8b-8192", "mis
tral-saba-24b", "gemma2-9b-it"])
else:
    model = st.sidebar.selectbox("Select Hugging Face Model", [
        "mistralai/Mixtral-8×7B-Instruct-v0.1",
        "meta-llama/Llama-3.2-11B-Vision-Instruct",
        "google/flan-t5-large"
    ])

# Adjust response parameters
temperature = st.sidebar.slider("Temperature", min_value=0.0, max_value=2.0,
value=0.7)
max_tokens = st.sidebar.slider("Max Tokens", min_value=50, max_value=800,
value=150)

# Main interface for user input
st.write("Go ahead and ask any question")
user_input = st.text_input("You:")

# Handle user input and API key validation
if user_input and api_key:
    try:
        response = generate_response(user_input, api_key, provider, model, tem
perature, max_tokens)
        st.write(response)
    except Exception as e:
```

```
        st.error(f"Error: {str(e)}")
elif user_input:
    st.warning(f"Please enter the {provider} API Key in the sidebar")
else:
    st.write("Please provide the user input")
```